

Improved update rule for probabilistic computers

Andrew Rockovich^{1,*}, Gregory Lafyatis,¹ and Daniel J. Gauthier²¹*Department of Physics, The Ohio State University, Columbus, Ohio, USA*²*ResCon Technologies, LLC, Columbus, Ohio, USA* (Received 1 April 2025; revised 30 May 2025; accepted 26 September 2025; published 4 November 2025)*This paper is a contribution to the Physical Review Applied collection titled [Physics-Inspired Computing](#).*

Many hard combinatorial problems can be mapped onto Ising models, which replicate the behavior of classical spins. Recent advances in probabilistic computers are characterized by parallelization and the introduction of novel hardware platforms. An interesting application of probabilistic computers is to operate them in “reverse” mode, where the network self-organizes its behavior to find the input bits that result in an output state. This can be used, for example, as a factorizer of semiprimes. One issue with simulating probabilistic computers on standard logic devices, such as field-programmable gate arrays, is that the update rules for each spin involve many multiplications, evaluation of a hyperbolic tangent, and a high-resolution numerical comparison. We simplify these rules, which improves the spatial and temporal circuit complexity when simulating a probabilistic computer on a field-programmable gate array. Applying our method to factorizing semiprimes, we achieve at least an order-of-magnitude reduction in the on-chip resources and the time to solution compared to recently reported methods. For a 32-bit semiprime, we achieve an average factorization in ~ 100 s. Our approach will inspire new physical realizations of probabilistic computers because we relax some of their update-rule requirements.

DOI: [10.1103/1wc-j7yp](https://doi.org/10.1103/1wc-j7yp)

I. INTRODUCTION

Modern digital computers have revolutionized nearly every aspect of society, leading to the information era. Their success is based on computer hardware that implements a small set of deterministic logic gates for universal computing and a suite of algorithms that run on this platform. Unfortunately, some important problems cannot be easily solved on a digital computer due to a lack of efficient algorithms [1]. By “efficient,” we refer to a problem that can be solved in a time that scales better than a superpolynomial function of its size and preferably as a low-order polynomial or better. Hard problems include optimization tasks such as simulating quantum systems, factoring large semiprimes, the traveling-salesman problem, binary satisfaction problems, etc.

The existence of hard problems motivates exploring other computing paradigms beyond the von Neumann architecture of digital computers. One promising approach is to use quantum mechanical spins to represent information, which can take advantage of quantum superposition and entanglement [2]. This concept gained great interest when algorithms that can solve a subset of hard “classical” problems efficiently were identified on a quantum computer [3,4]. As with digital computers, a few discrete quantum gates can be composed to realize universal quantum

computation. Considerable worldwide effort is underway to achieve fault-tolerant quantum computers [5–7].

A parallel effort is underway to determine whether networks of emulated atoms interacting via their magnetic dipole moments—classical spins—can offer an advantage for solving hard problems [8–11]. They are typically called Ising machines and are studied using the theory of statistical mechanics. Problems are encoded in the interactions between the networked spins, and the solution corresponds to the energy ground state of the network. A properly designed network self-organizes its dynamics to seek out the ground state, as illustrated in Fig. 1(a). As with classical digital and quantum computers, a small number of gates can be composed to realize universal computing.

Ising machines are characterized by a temperature that perturbs the spin orientation. These perturbations help the system find the global ground state and prevent it from being stuck in a local energy minimum. Typically, the system starts at a higher temperature and is cooled slowly in a process known as annealing, analogous to optimization algorithms running on classical computers such as simulated annealing [8]. More sophisticated optimization routines include parallel tempering [12] and simulated bifurcation [13,14]. A variety of devices, such as photonic oscillators [15], magnetic tunnel junctions [16–18], and memristors [19], are being developed to realize scalable Ising machines. Because of their inherent noisy behavior, these machines are sometimes referred to as probabilistic

*Contact author: rockovich.6@osu.edu

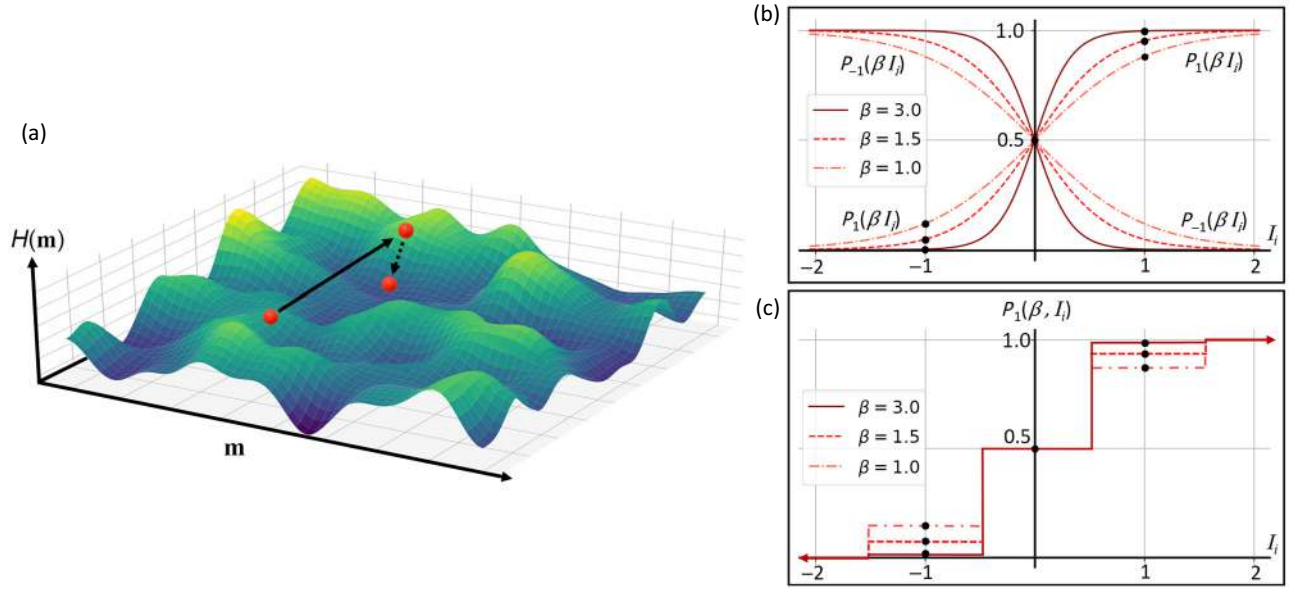


FIG. 1. A P-bit network finds an energy minimum. (a) A simplified illustration of the energy landscape, where the horizontal dimensions are a flattened N -dimensional $\mathbf{m}$. The red balls are the energies of given states, the solid line is an energetically unfavorable spin flip, and the dashed line is an energetically favorable flip. (b) Typical update rules' continuous spin-flip probabilities. (c) Our discretized and truncated spin-flip probabilities.

computers, which we adopt here, and the fundamental carrier of information is a probabilistic bit or P-bit.

One interesting aspect of P-bit networks is that they can operate in both the forward and reverse modes. In the forward mode, the P-bits corresponding to the “input” bits of the system are clamped at the input values, and the “output” P-bits of the network self-organize to provide the output solution. This is the standard operation mode of digital logic circuits. In the reverse mode, the output P-bits are clamped, and the network self-organizes so that the input P-bits take on values consistent with the output states. The reverse mode of a probabilistic multiplier circuit can be used, for example, to find the factors of a semiprime, where the output P-bits encode the semiprime, and the network is driven to its ground state with the input P-bits encoding the two factors.

The primary purpose of our paper is to demonstrate that the P-bit update algorithm for a probabilistic computer can be greatly simplified using the method illustrated in Figs. 1(b) and 1(c). We approximate the typical sigmoidal probabilistic update rules by a step function over integer input values. Then, we truncate the update rules and sparsify the network [20]. This enables us to eliminate all computationally expensive clocked arithmetic that is typically required for P-bit updates, converting them to a much more efficient lookup-table format. This result greatly speeds up the network dynamics and reduces the hardware resources needed to emulate a probabilistic computer on complementary metal oxide semiconductor (CMOS) devices. It may also suggest other P-bit hardware solutions.

In the next section, we introduce the well-established two-body-interaction-based P-bit update rules and show how a P-bit network can be composed to solve hard problems. We introduce our new update rule in Sec. III. We apply our approach to operate the fundamental AND and full-adder gates in reverse mode in Sec. IV. We emulate a P-bit network using a field-programmable gate array and demonstrate high-accuracy operation with fewer resources than in previous work. We then show how to realize a multiplier operating in reverse mode, which can be used for semiprime factorization. We demonstrate high-accuracy factorization of 32-bit numbers (16-bit $\times$ 16-bit factors) in Sec. V. As reported recently by other groups, we observe that the time to solution for this problem scales exponentially with the number of bits. In Sec. VI, we compare our results with the current state of the field and look toward the future.

II. BACKGROUND

The lattice of spins is constructed out of ferromagnetic (antiferromagnetic) interacting adjacent P-bits (state m_i in the bipolar range $\{-1, 1\}$) via positive (negative) interaction and described by the Hamiltonian

$$H(\mathbf{m}) = - \sum_i h_i m_i - \sum_{i < j} J_{ij} m_i m_j. \quad (1)$$

Here, the weights form a symmetric $(N \times N)$ matrix $\mathbf{J}$ associated with quadratic interactions, and N is the system

size (total number of P-bits). In addition, each P-bit may be biased toward spin “up” (1) or “down” (−1) described by the $(N \times 1)$ column vector $\mathbf{h}$.

For a network at a pseudo-inverse-temperature β , Eq. (1) yields the familiar Boltzmann probabilities

$$P(\mathbf{m}) = \frac{\exp[-\beta H(\mathbf{m})]}{\sum_{\mathbf{c}} \exp[-\beta H(\mathbf{m}_{\mathbf{c}})]}, \quad (2)$$

where the denominator is the partition function, a sum over all possible state combinations. As $\beta \rightarrow \infty$ (pseudo-temperature $\rightarrow 0$), only the P-bit configurations with the lowest energy will have a nonzero probability. To find the global energy minimum, β varies in time. It starts “hot” initially (say, $\beta = 1$), and then increases toward ∞ (zero pseudo-temperature) with a predefined waveform [8].

The interaction weights can be designed directly using linear programming [21]. The constraints to the linear program are: entries appearing in the circuit truth table take on the lowest energy, and those that are not in the truth table have a larger energy. It is common to use integer-constrained linear programming so that all elements of $\mathbf{J}$ and $\mathbf{h}$ are integers. For larger circuits, $\mathbf{J}$ and $\mathbf{h}$ can be found by adding those of the smaller circuits. With these interactions, the P-bits network self-organizes to find the solution to the problem: the configurations that appear in the circuit truth table.

Recent efforts show that sparse Ising machines scale better with problem size than those with all-to-all connectivity [22,23]. Additional sparsification provides further potential benefits by reducing the number of conflicting assignments [20,22]. This allows for simultaneous updating of large blocks of P-bits, greatly speeding up the energy minimization process. Sparsification is done by adding auxiliary COPY gates to the circuit, which we illustrate in the Appendix. Identifying simultaneous update blocks is done using a greedy graph-coloring algorithm. The different color blocks are updated sequentially using phase-shifted clocks so that every P-bit is updated in a single clock cycle.

The Hamiltonian in Eq. (1) specifies the energy landscape conceptually illustrated in Fig. 1(a). A “particle” on this energy landscape will seek to lower its energy by moving in the direction of the negative gradient. Typically, the P-bits are updated one at a time, familiar from stochastic gradient-descent algorithms [24], to prevent simultaneous flips of adjacent P-bits that create a contradiction. For updating one at a time, the update rule for each P-bit is found from

$$I_i = -\frac{\partial H}{\partial m_i} = \sum_j J_{ij} m_j + h_i, \quad (3)$$

where I_i is the directivity, the propensity to make P-bit i equal to 1 (up, $I_i > 0$) or −1 (down, $I_i < 0$) on the update step.

When the network is at a finite pseudo-temperature, the spins are perturbed so that the update rule is not exact: There is a chance that the update will cause the P-bit to move in an energy-increasing direction. As mentioned above, these perturbations are needed to prevent the system from getting stuck at a local minimum. In this case, the update rule is usually written as [25]

$$m_i = \text{sgn}[\tanh(\beta I_i) - \text{rand}(-1, 1)], \quad (4)$$

where the hyperbolic tangent confines the directivity to the range $(-1, 1)$,

$$\text{sgn}(x) = \begin{cases} -1, & x < 0, \\ 0, & x = 0, \\ 1, & x > 0, \end{cases} \quad (5)$$

and rand is a uniformly distributed random number in the range $(-1, 1)$ representing the perturbation.

When $\tanh(\beta I_i) = 0$, which is when $I_i = 0$ independently of β , the P-bit state is scrambled by the rand function. For $|\beta I_i| \gg 1$, the hyperbolic tangent function is saturated, and the rand function has almost no effect on the state. Overall, the probability that the P-bit attains the state ± 1 is given by

$$P(m_i = \pm 1) = \frac{1}{2}[1 \pm \tanh(\beta I_i)]. \quad (6)$$

Figure 1(b) shows the conditional probabilities as a function of I_i for different β .

In recent realizations of P-bit networks on CMOS-based field-programmable gate arrays (FPGAs) [20,26], Eq. (4) for each P-bit is evaluated as faithfully as possible, with a typical circuit shown in Fig. 2. Here, the state of each P-bit input to P-bit i is multiplied by the interaction weights using a finite-precision multiplier, the values are added

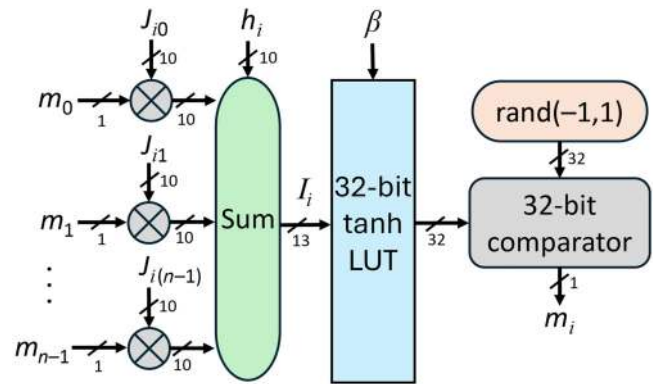


FIG. 2. Previous P-bit simulation circuits. Typical circuitry required on an FPGA to update the i th P-bit. The gray circles are multipliers, and the bit widths of each value are given. (Adapted from Ref. [20].)

and combined with the bias, and the hyperbolic function is applied by a β -dependent 32-bit lookup table (LUT). The result is compared to a 32-bit pseudo-random-number generator that spans the range $(-1, 1)$, which outputs the P-bit state (either -1 or 1).

This circuit requires fast and slow clocks. It takes multiple fast clock cycles to execute the P-bit circuit and slower clocks to perform the sequential updates of the P-bit in the network. Furthermore, the circuit consumes substantial on-chip resources. In the next section, we describe methods to drastically reduce its complexity and speed up its operation.

III. AN EFFICIENT P-BIT UPDATE RULE

There are several steps we take to simplify the update rule in Eq. (4). The first is to rewrite the update rule as

$$m_i = \text{RNG}(b) \text{sgn}(I_i), \quad (7)$$

where the *biased random-number generator* is defined by

$$\text{RNG}(b) = \begin{cases} +1, & \text{with probability } b, \\ -1, & \text{with probability } (1 - b), \end{cases} \quad (8)$$

and the bias is in the range $(0.5, 1)$ and given by

$$b = \frac{1 + |\tanh(\beta I_i)|}{2}, \quad (9)$$

where $b = 1/2$ for an unbiased random-number generator and $b = 1$ for a fully biased bit [that is, $m_i = \text{sgn}(I_i)$].

With these definitions, the P-bit update is governed by the sign function in Eq. (4), independently of β , and by a bit flip governed by the bias, which depends on β . We show below that decomposing the update rule in this way greatly reduces the P-bit circuit complexity.

The second step is to realize that I_i is an integer when the components of $\mathbf{J}$ and $\mathbf{h}$ are integers. Most values of I_i saturate the hyperbolic tangent function in Eq. (9) when β is not too small (say, $\beta \geq 1$). This means that the hyperbolic tangent function needs to be evaluated only for a few of the smaller values of I_i indicated by the dots in Fig. 1(b). To stress this point, we redraw the probabilities as step functions in Fig. 1(c). We then need to evaluate b for a few values of β to achieve fast convergence to the ground state of the P-bit network.

The final step is to realize that $\text{sgn}(I_i)$ can be found using a LUT, which matches the architecture of FPGAs typically used to simulate P-bit networks. For P-bit i with M connections, there are 2^M entries in the LUT, which becomes impractical when M is large. However, the sparsification of $\mathbf{J}$ greatly helps in this regard, which is already done to allow for block updating of the network. In particular, we follow Aadit *et al.* [20] and set $M \leq 5$. Another advantage of this approach is that the LUT compares only binary

values as opposed to the 32-bit comparator and the LUT used for the high-precision hyperbolic tangent in the circuit shown in Fig. 2.

Continuing with the analysis of our approach, the probability of an energy-increasing P-bit assignment is only $\sim 0.5\%$ when $I_i = \pm 2$ and $\beta \sim 1.5$. Based on this observation, we make a further simplification for the bias:

$$b = \begin{cases} (1 + |\tanh(\beta I_i)|)/2, & |I_i| \leq 1, \\ 1, & |I_i| > 1. \end{cases} \quad (10)$$

We find that this provides enough randomness to allow the network to escape local energy minima.

It is customary to translate the bipolar spin states in the range $\{-1, 1\}$ to binary states in the range $\{0, 1\}$ because this range matches the traditional description of CMOS logic gates. We translate the P-bits states as [20]

$$s_i = \frac{m_i + 1}{2}, \quad (11)$$

and the weight matrices as

$$\mathbf{J}_{\text{binary}} = 2\mathbf{J}, \quad (12)$$

$$\mathbf{h}_{\text{binary}} = \mathbf{h} - \mathbf{J}\mathcal{A}, \quad (13)$$

where $\mathcal{A}$ is an $(N \times 1)$ vector of 1's. For the rest of the paper, we use the binary convention and drop the “binary” subscript.

The remaining part of our new update rule is to describe an efficient method for the biased random-number generation. The goal is to realize a sequence that is mostly 0 (binary range) with probability b and 1 with probability $1 - b$. This can be accomplished by performing the logical R -input AND operation on R independent and unbiased RNGs. The probability of obtaining a 1 is then

$$b = \begin{cases} 1/2, & I_i = 0, \\ 1/2^R, & |I_i| = 1, \\ 1, & |I_i| > 1. \end{cases} \quad (14)$$

We find that choosing one value of R is enough to obtain fast convergence to the ground state; that is, no annealing is needed.

In our implementation below, we feed the R -input AND gate with randomly selected bits from 46-bit linear feedback shift registers (LFSRs), which is a pseudo-random-number generator with a repeat pattern every 2^{46} bits. This is much larger than the number of P-bit update steps needed for our experiments. Each color requires a set number of LFSRs, each contributing 46 random bits, depending on the number of P-bits in that color, whether they contain cases of $I_i = \pm 1$ or $I_i = 0$, and R . The maximum number of LFSRs we require for a single color is for a color

in the 16-bit $\times$ 16-bit multiplier in Sec. VB with 608 P-bits, requiring 39 of the 46-bit LFSRs. We select R at random (without replacement) and AND them to generate one biased bit. Unbiased bits are drawn from the same set without replacement.

IV. INDIVIDUAL LOGIC GATES

For small probabilistic circuits, we find $\mathbf{J}$ and $\mathbf{h}$ directly with linear programming as described in Sec. II, without auxiliary bits or sparsification. We consider the AND and full-adder (FA) gates because they are the components of a binary multiplier described in the next section. We also only show experimental results for the reverse mode of the P-bit network because the forward mode can be found using standard logic. The experiments are conducted on a Xilinx Zynq Ultrascale+ RFSoc 4×2 FPGA (XCZU48DR-2FFVG1517E) with 930k system logic cells, and 425k configurable logic blocks (CLBs) consisting of six-input LUTs and flip-flops (FFs). Data are collected from the P-bit network using the on-chip integrated logic analyzer (ILA) and transferred to a laptop computer via the JTAG (Joint Test Action Group) interface for offline analysis.

A. The AND gate

The probabilistic AND gate is a three-P-bit logic gate illustrated in forward and reverse modes in Fig. 3(a), and the weights and biases are given in Fig. 3(b). When operating the AND gate in forward mode, the input P-bits A and B are clamped to the desired input values, and the output P-bit C changes its value to find the minimum energy of the network. Randomness is never needed for the forward AND gate: $|I_i| > 1$ in every line of the LUT seen in Fig. 3(c), so the updates to C are deterministic.

The situation is more complex when the probabilistic AND gate is operated in reverse mode. There are two cases to consider: $C = 0$ and $C = 1$. When $C = 1$, the only possible state consistent with the AND truth table is $A = 1$ and $B = 1$. Figures 3(d) and 3(e) show the update rules for P-bits A and B , respectively, when $\beta \rightarrow \infty$. We see that I_i takes on large values for $C = 1$, and hence we do not apply randomness using the approximate update rule in Eq. (14).

When $C = 0$, we have three possible input configurations: $\{A, B\} = \{0, 0\}$, $\{0, 1\}$, and $\{1, 0\}$, which all have the same (ground-state) energy and should be equally likely. When updating A , the update is deterministic if $\{B, C\} = \{1, 0\}$ for $I_i = -2$. On the other hand, A (B) is set to an unbiased random number when $\{B, C\} = \{0, 0\}$ (when $\{A, C\} = \{0, 0\}$) because $I_i = 0$.

We update A and B sequentially at 400 MHz on the positive edges of a single clock with two outputs shifted by 120° . Once A and B update, we measure them on the positive edge of a third clock output with an additional 120° phase shift. Thus, full updating of the network and reading out its state happens in a single clock cycle. We start the system in state $\{A, B\} = \{0, 0\}$. When started in a higher-energy state, the first updated P-bit puts the system into an allowed state after the first P-bit is updated.

Figure 4 shows the experimental results for the probabilistic AND gate operating in reverse mode with $C = 0$. We see that the network visits all three states with nearly equal probabilities, as expected based on our discussion above regarding the Boltzmann probabilities given by Eq. (2).

Based on the clock frequency, we perform 8.0×10^8 P-bit flips per second (FPS). The ILA is likely the limiting factor for the clock frequency, as this speed is around the usual maximum before timing violations. The on-chip resources for the AND P-bit network are 10 LUTs and 14 FFs, not including the ILA used for data collection, which consumes an additional 1036 LUTs and 1827 FFs.

B. The full adder

The FA is a five-P-bit gate shown in Fig. 5(a), with weights and biases given in Fig. 5(b). In forward mode, the three input bits $\{A, B, C_{in}\}$ are clamped to input values, and the system self-organizes to produce the 2-bit sum $\{S, C_{out}\}$. In the forward mode of operation, there is a single entry in the truth table corresponding to each input.

In reverse mode, clamping $\{S, C_{out}\} = \{0, 0\}$ (or $\{1, 1\}$) results in a single state $\{A, B, C_{in}\} = \{0, 0, 0\}$ (or $\{1, 1, 1\}$) at the energy minimum, and no randomness is needed to drive the network to these states.

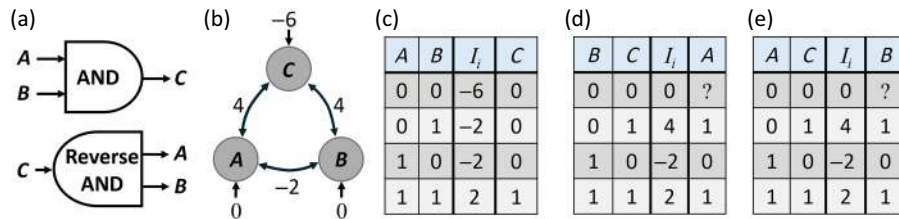


FIG. 3. The AND gate. (a) Typical circuit depiction. (b) Probabilistic formulation, with weight and bias terms in binary form. (c)–(e) LUTs for updating output P-bit (c) C , (d) A , where the ambiguity (?) corresponds to equal probability of $A = 0$ and $A = 1$, and (e) B , which is symmetric to (d).

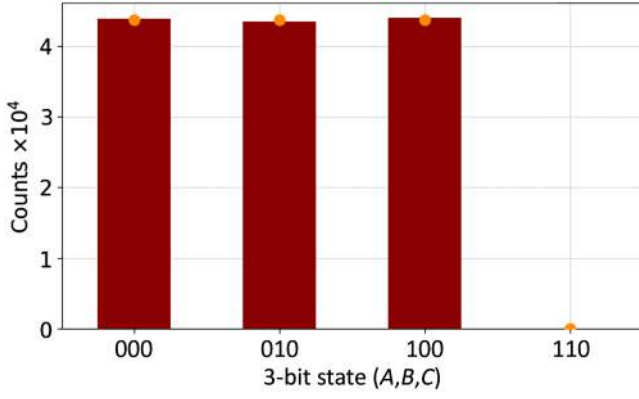


FIG. 4. The AND gate experiments. Frequency diagram when running the AND gate in reverse mode, with P-bit C clamped to 0, sampled after each of the 131 072 full system updates. The orange dots show the Boltzmann probabilities given by Eq. (2).

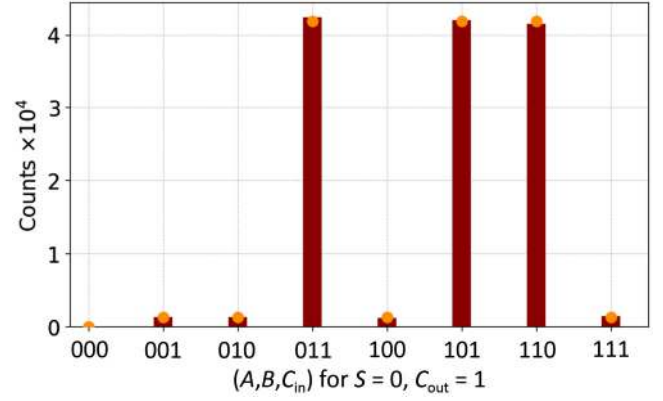


FIG. 6. Full-adder experiments. Frequency diagram for the FA operating in reverse mode with $S=0$ and $C_{out}=1$ for the 131 072 full system updates. The orange dots show the Boltzmann probabilities given by Eq. (2).

There are multiple energy-minimum states when clamping $\{S, C_{out}\} = \{0, 1\}$ or $\{1, 0\}$. For $\{S, C_{out}\} = \{0, 1\}$, these are $\{A, B, C_{in}\} = \{1, 1, 0\}$, $\{1, 0, 1\}$, and $\{0, 1, 1\}$, which should be visited with equal probability. Unlike the AND P-bit network, each energy-degenerate FA state resides in an isolated energy minimum. Transitions between them require an energy-increasing P-bit update provided by the randomness.

An example sequence of the P-bit updates with their corresponding energy is shown in Fig. 5(c). Without an energetically unfavorable flip, the input P-bits will remain in a single state. Without an energy-increasing update (due to the randomness), the state will stay in $\{1, 0, 1\}$ forever. This illustrates the need for probabilistic updates.

Figure 6 shows the experimental results for the probabilistic FA gate operating in reverse mode with $\{S, C_{out}\} = \{0, 1\}$. The circuit visits each global energy minimum with nearly equal frequency. We update A , B , and C_{in} sequentially at 350 MHz on the positive edges of the multiple clock outputs, each phase-shifted by 90° . We measure the state on the positive edge of a third, equally phase-shifted, clock using the ILA. For the cases when $|I_i| = 1$, we use

$R = 5$ to generate b in Eq. (14), which is equivalent to $\beta = 1.717$ by comparing the flip probability of $I_i = \pm 1$ cases. We also give the expected number of counts at each state given by the Boltzmann probability distribution from Eq. (2). We find that we obtain the expected Boltzmann probability distributions for a wide range of the bias parameter b , and hence we hold it constant.

The input P-bit state is initialized to $\{A, B, C_{in}\} = \{0, 0, 0\}$. It immediately transitions to an allowed state after updating the first P-bit. The $\{0, 0, 0\}$ state is never revisited because it requires updates not supported by Eq. (14). Similarly, for $S = 1$ and $C_{out} = 0$, $\{A, B, C_{in}\} = \{1, 1, 1\}$ is never visited.

For the FA, we perform 1.05×10^9 P-bit FPS. We do not maximize the clock frequency for this small probabilistic gate, and the lower clock frequency used here is also likely due to the somewhat larger ILA required to collect data from this network. The on-chip resources for this network are 27 LUTs and 36 FFs, not including the ILA for data collection, which consumes an additional 1058 LUTs and 1848 FFs.

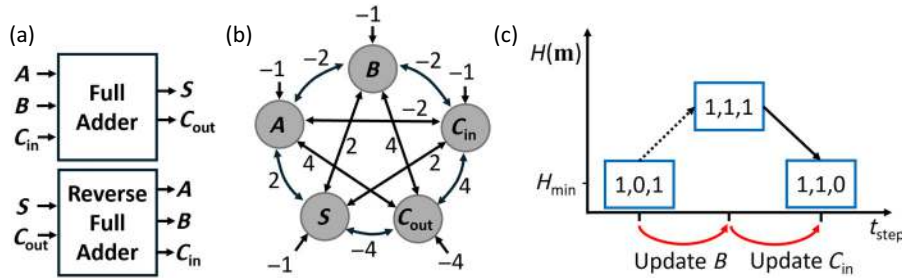


FIG. 5. The full adder. (a) Typical circuit diagram. (b) Probabilistic formulation, with weights and biases in binary form. (c) Example input (A, B, C_{in}) state evolution in reverse mode, initial condition $= \{1, 0, 1\}$, outputs clamped to $\{S, C_{out}\} = \{0, 1\}$. Dotted line is energetically unfavorable flip (with $I_i = \pm 1$). Solid lines are energetically favorable flips.

V. FACTORING SEMIPRIMES

We use conventional binary long multiplication to construct a P-bit network that can be run in reverse mode to factorize semiprimes. The binary partial products are found using AND gates, which are summed using FAS as described in the [Appendix](#).

We construct the full-circuit interactions and biases by combining the **J** matrix and **h** vector for the AND and FA gates shown in Figs. 3(b) and 5(b). We sparsify the circuit following the procedure described in Aadit *et al.* [20] and summarized in the [Appendix](#). For a sparsified k -bit $\times$ k -bit multiplier with a maximum of five connected neighbors ($M \leq 5$), there are k^2 AND gates, $k(k-1)$ full adders, and $5k(k-1) - 3k$ COPY gates between them. This expression includes COPY gates between FASs, and between FASs and ANDs. Each input bit is distributed to k AND gates. We add $r = 1 \rightarrow \log_{M-1}(k)$ layers of P-bits to sparsify each input bit, as shown in Fig. 11 in the [Appendix](#). Layer l_0 consists of the k input P-bits distributed to AND gates. Each l_r adds an additional

$$l_r = \text{ceil} \left[\frac{l_{r-1}}{M-1} \right] \quad (15)$$

P-bits, where the ceiling function rounds its argument to the next highest integer. Sparsifying the input P-bit distribution also adds l_r additional COPY gates per layer, excluding the top layer, which will always be $l_r = 1$. The P-bit at the top layer is the one we measure when running the multiplier in reverse mode. We also require R random bits for each P-bit with an instance of $|I_i| = 1$, and one random bit for each P-bit with an instance of $I_i = 0$. We divide these by color, and generate $\text{RNG}_{\text{total}, \text{col}}/46$ of the 46-bit LFSRs for color col . Random bits are drawn at random from the entire list of $\text{RNG}_{\text{total}, \text{col}}$ random bits without replacement to form b in Eq. (14).

Below, we consider multipliers from 3-bit $\times$ 3-bit up to 16-bit $\times$ 16-bit. We use the ILA for data collection as we did in Sec. IV. We add the Xilinx virtual input-output (VIO) IP block to change products and reset the system, resulting in a general-purpose factorizer. We find the best time to solution when using $R = 3$ ($R = 4$) for the biased random-number generator for $k < 16$ ($k = 16$). We selected R by minimizing the time to solution using a grid search. The optimal R presents the best balance between rapidly exploring the solution space (requiring more noise, better with smaller R) and finding energy minima (requiring less noise, better with larger R). We use a larger $R = 5$ for the 3×3 multiplier analysis to increase the ratio of probability to be in allowed states to that of unallowed states.

A. Small multiplier

The optimization energy surface becomes increasingly complicated as the problem scale increases, such as for

a multiplier composed from many smaller gates. This requires additional search time to identify the global minimum. We clamp the output bits to the semiprime to be factored and randomly initialize the remaining network P-bits. We then execute 1000 full system updates and read the input P-bits corresponding to the two prime-factor candidates. This value is unrelated to the number of updates typically required to find a solution. For any more than 1000 full system updates, state visitation statistics change little, indicating that 1000 updates are enough for the system to settle into deeper energy minima. We repeat this sequence 131 072 times.

A full system update involves updating all the P-bits that share a color simultaneously and updating the colors sequentially. Each color updates on a separate phase-shifted clock edge. The LFSRs that generate the RNGs for one color are updated simultaneously on the previous color's clock edge. We use the mixed-mode clock manager (MMCM) block on the FPGA to realize parallel 110-MHz clocks, using the 100-MHz low-voltage differential signaling reference clock as input. The result is a set of highly stable, synchronized, low-phase-noise clocks that result in predictable system behavior. We add one additional equally phase-shifted clock (from the MMCM) that enables state readout after each full system update. We consider P-bit multiplier networks with $M = 5$, so we have six 110-MHz clocks separated by a 60° phase shift between each. Every P-bit is updated in a single clock cycle.

Figure 7 shows the “input” P-bits (factors) state statistics for the output clamped to $10_{10} = 001010_2$, where the subscript denotes the number base. Scaling the vertical axis in Fig. 7 by the total number of samples transforms it into the state probability distribution. We note that the input state frequency statistics disregard the states of the auxiliary P-bits. The correct solutions are $2_{10} \times 5_{10} = 010_2 \times 101_2$ or $5_{10} \times 2_{10} = 101_2 \times 010_2$. The horizontal axis indicates the factors in the format of the first factor concatenated

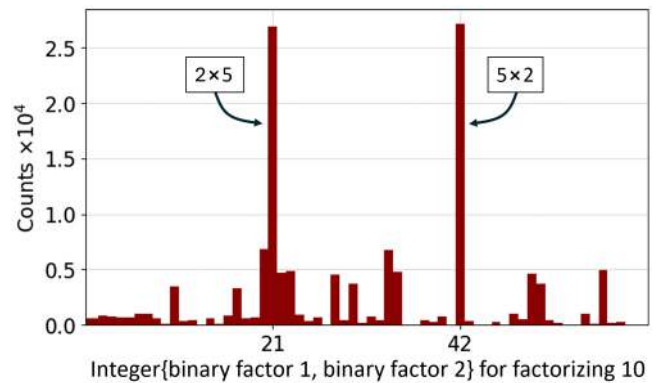


FIG. 7. Small multiplier experiments. Frequency diagram for the input bits of a 3×3 multiplier run in reverse mode when the output bits are set to 10 in base 10.

by the second factor in binary format; then this number is converted to base 10. Using this convention, solutions are $010101_2 = 21_{10}$ and $101010_2 = 42_{10}$. These solutions correspond to the tallest two bars.

The smaller peaks in the plot have contributions from two cases: either the system had an energy-increasing update just before the sample, or the state is in a local minimum. Sampling after 1000 updates minimizes contributions from local minima to the state population histogram, giving us a clearer picture of the energy landscape. Frequently visited nonsolution states correspond to deeper energy minima that require more simultaneous energetically unfavorable flips to escape.

For the 3×3 multiplier, we perform 5.94×10^9 P-bit FPS. We calculate this value by taking the total number of P-bits in the network (63), subtracting those that are clamped [outputs (6) and the zero-input P-bits to the FAS (3); see the [Appendix](#)], and multiplying the result by clock frequency. We do not maximize the clock frequency for this smaller multiplier, instead keeping it the same for all k . The lowest clock frequency is for $k = 16$ discussed below. The on-chip resources for this network are 131 LUTs and 345 FFs, not including the ILA for data collection or the VIO for changing memory cells for the semiprimes, which together consume an additional 1597 LUTs and 2693 FFs.

B. Larger multipliers

We study the time to solution for multipliers running in reverse mode up to $k = 16$, showing significant ($\gtrsim 1$ order-of-magnitude) improvements over similar approaches. Similarly, we show a > 1 order-of-magnitude decrease in the number of on-chip resources required to construct multiplier circuits. For each, we keep the biased random-number generator on all the time for cases of $I_i = \pm 1$, with no annealing schedule. When the correct solution is found, some P-bits in the network have $I_i = \pm 1$, and hence the system is eventually driven out of the global minimum once there is a bit flip that increases the energy of the state.

For this reason, we add an oracle that checks whether factors match the product state after each full system update using on-chip dedicated multipliers on the FPGA. Once a solution is found, the circuit reports the number of full system updates, sets the unclamped P-bits to random initial conditions, and restarts the search. We typically repeat this process 1024 times. We perform fewer trials for the 16-bit $\times$ 16-bit multiplier because of the longer time to solution, but we never use fewer than 13. We repeat this process for 10 semiprimes for each multiplier size. The semiprimes are constructed by arbitrarily choosing k -bit prime integers, avoiding repetition. Repetition is unavoidable for $k = 6$ because there are only seven prime integers in this case.

As we did for the smaller ($k = 3$) multiplier in Sec. [V A](#), we use $M = 5$ for all multipliers here. We perform updates for all multipliers (all k) using 110-MHz clocks, identical to those we used for the $k = 3$ multiplier. Because the clock frequencies do not change, the data we present in terms of time to solution can be converted to iterations to solution by multiplying by 110 MHz. For $k = 6$ –14, the choice $R = 3$ results in the most efficient time to solution owing to its rapid exploration of the solution space. For $k = 16$, we find that $R = 4$ minimizes time to solution.

The $k = 16$ multiplier performs 2.29×10^{11} FPS (2128 total P-bits, 32 clamped to the product state, and 16 clamped to zero for 2080 P-bits updated every clock cycle), with as many as 6.69×10^{10} FPS for one color. As Aadit *et al.* [20] find, the flips per second grow linearly with problem size. We estimate that they observe 4.33×10^{10} FPS in their $k = 16$ multiplier based on the scaling linearity in combination with other provided data. Our observed speedup comes from the increased clock rate.

Figure 8 shows the scaling of the average time to solution (blue dots and line of best fit) as a function of $2k$ (the number of bits in the semiprime) on a semilogarithmic scale (left vertical axis). The data are fit to a straight line, resulting in the expression of best fit: $(10^{-8} \text{ s}) \times \exp(2k/1.5)$. Figure 8 also shows the total number of each on-chip resource used to realize each multiplier circuit plotted on a linear scale (right vertical axis). The scaling is quadratic, as expected from the quadratic scaling of the number of P-bits with multiplier size. The line of best fit for LUTs is $3.24 \times (2k)^2 + 30$, and for FFs is $9.59 \times (2k)^2 + 1$. The resource utilization data exclude resources used by the ILA for data collection and the VIO for changing semiprimes, but include the oracle for indicating that a solution has been found.

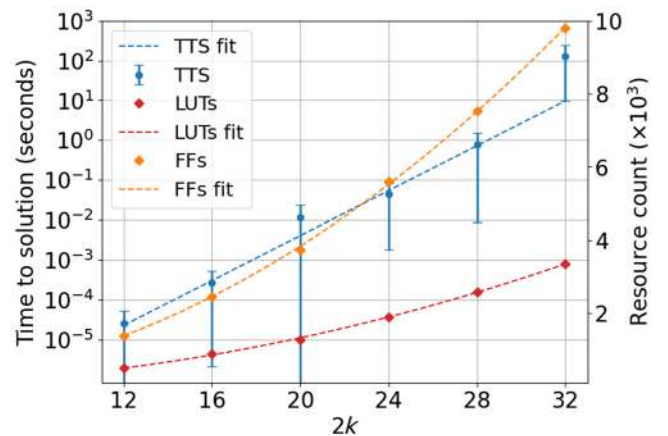


FIG. 8. Performance and on-chip resources of the multiplier circuits running in reverse mode. The average time required to factorize different product sizes (blue dots and dashed line), and the number of LUTs (red diamonds and dashed line) and FFs (orange diamonds and dashed line) of the circuit.

TABLE I. Chip area. Resource usage between our 16-bit $\times$ 16-bit factorizer and an unsparisified 16-bit $\times$ 16-bit factorizer with the same underlying architecture.

Resource	Typical CMOS method ^a	Our method
LUTs	62 969	3350
Flip-flops	9393	9798

^aFrom Onizawa *et al.* [21].

The exponential scaling of the time to solution as a function of problem size demonstrates that the P-bit network is not an efficient semiprime factorizer. However, the observed scaling for our architecture and absolute time to solution are better than reported by Aadit *et al.* [20]. They find a best fit of $(10^{-7.17} \text{ s}) \times \exp(2k/1.26)$, which has a larger prefactor and a larger exponent compared to ours. They also report an average time to solution of over 10^3 s for $k = 16$, which is an order of magnitude larger than our result.

Our approach also uses fewer on-chip resources. The sparsified architecture we use here is identical to that of Aadit *et al.* [20], but they use the P-bit circuit shown in Fig. 2. Unfortunately, they do not report their chip resource usage. Table I compares our chip usage to a similar long-multiplication architecture but with a dense (unsparisified, $M \gg 1$) connectivity from Onizawa *et al.* [21]. Even though we increase the total P-bits and the number of nonzero weights, we use over an order-of-magnitude fewer on-chip LUTs and a comparable number of FFs. We exclude ILA for data collection and VIO for altering the semiprime from the resource comparison. Together, these auxiliary resources consume an additional 4558 LUTs and 4875 FFs. Their unsparisified architecture does not allow for graph coloring, massively parallel P-bit updates, and our approach of using small LUTs for the update rule, and hence we expect that their time to solution will be substantially longer.

VI. CONCLUSIONS

Our experiments demonstrate that new P-bit update rules can greatly simplify the circuit used to simulate P-bit network dynamics. We focus on running the network in the reverse model because standard CMOS logic can be used to find the solution in the forward mode. We find that the time to solution and the on-chip resource usage is reduced substantially, sometimes by orders of magnitude, compared to the state of the art reported in the recent literature. Using an FPGA with our new update rule, a sparsified circuit, and graph coloring allows for massively parallel execution of the search.

It is not surprising that we obtain an exponential scaling of the time to solution for factoring semiprimes because

we are performing a simulation of a P-bit network using clocked logic. The P-bit network is effectively performing a multidimensional energy-minimizing search with occasional random bit flips. This is similar to other approaches to this problem, such as basin hopping [27], which also has an exponential scaling with problem size.

Every multiplier we constructed and tested in Sec. V is a general circuit with resettable outputs (the semiprime) when operating in reverse mode by loading different bit values into on-chip memory. Also, every k -bit $\times$ k -bit multiplier can be used for $(\leq k)$ -bit $\times$ $(\leq k)$ -bit problems with similar time-to-solution scaling as in Fig. 8.

Recent P-bit research considers higher-order interactions [23,28–30]. That is, the Hamiltonian in Eq. (1) has terms that include cubic, quartic, etc. monomials of the spins. The problem has a compact form, where many spins are grouped into a simplex structure, resulting in a simpler energy landscape. However, there is a limit to the size of a simplex because we must store the interaction weights, and we require many logic bits on an FPGA to represent the higher-order spins. Further research is needed to understand the trade-off of these approaches.

Our approach is not limited to integer factorization. We expect that many probabilistic hard combinatorial problem solvers would benefit from the simplified update rules, such as the Max-Cut or 3-SAT problems. Our simplifications can also be introduced to probabilistic and/or stochastic machine learning algorithms for fast and accurate training [31].

We close by mentioning that our new update rule may suggest new analog approaches to creating P-bit networks and potentially better scaling with problem size. Much of the P-bit literature focuses on P-bit networks realized with magnetic tunnel junctions because they have a graded response with input current and have thermal noise [16–18]. Because we no longer need a graded response directly in the P-bit structure [the hyperbolic tangent function inside the sgn function of Eq. (3)], other physical devices might be good P-bit candidates, such as using autonomous logic (CMOS logic operated without a clock) [32].

ACKNOWLEDGMENTS

We gratefully acknowledge the financial support of the Air Force Research Lab (AFRL) under Agreement FA8650-19-1-1741 and conversation about this research with the members of the Ohio State University Center for Enabling Cyber Defense in Analog and Mixed Signal Domain (CYAN).

DATA AVAILABILITY

The code and data that support the findings of this article are openly available [33,34].

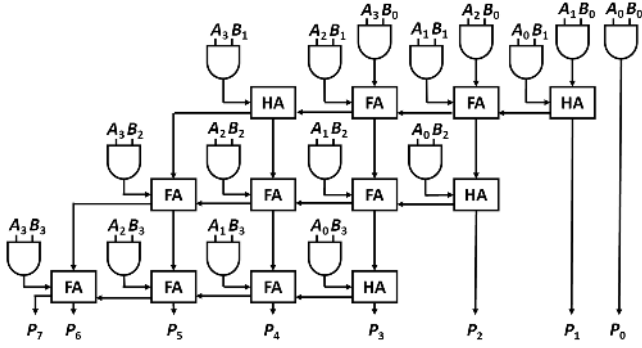


FIG. 9. Binary multiplier. Conventional 4-bit $\times$ 4-bit multiplier circuit. Input factors A (most-significant bit A_3 , least-significant bit A_0) and B , output product C . Each line and each input and output bit represents a P-bit in the system.

APPENDIX: MULTIPLIER STRUCTURE

As mentioned in the main text, we use a binary long-multiplication architecture, with an example of a 4-bit $\times$ 4-bit multiplier shown in Fig. 9. The three fundamental gates are the AND, half-adder (HA), and FA. To simplify the software that automatically generates $\mathbf{J}$ and $\mathbf{h}$ for larger multipliers, we substitute the HA with FA and clamp the input carry bit to zero, which produces the same function. These bits are ultimately removed by the Xilinx circuit compiler and optimizer (Vivado system) because they do not contribute to the system.

To sparsify the circuits, we add COPY gates between the AND and FA gates, and between each of the FAs, as shown in Fig. 10. We use a maximum of five nearest neighbors for each P-bit ($M \geq 5$), which aligns well with splitting P-bits that act as a bridge between FA gates.

The remaining dense part of the circuit is fanning the input P-bits to the AND gates: Each input P-bit must be distributed to k AND gates. To reduce the number of connections, we use a hierarchy of COPY gates. Figure 11 shows an example of this hierarchy for an 8-bit $\times$ 8-bit

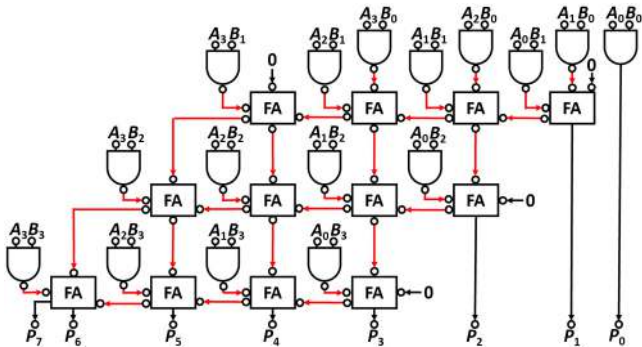


FIG. 10. Sparsified binary multiplier. A sparsified 4-bit $\times$ 4-bit multiplier, where P-bits are black open circles and COPY gates are red lines. We replaced HA gates with FA gates with C_{in} clamped to 0.

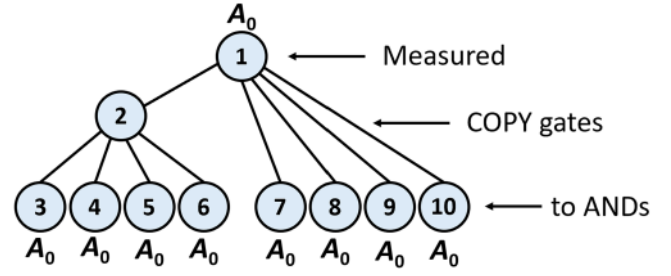


FIG. 11. Sparsifying fanout of the input P-bits to the network. Sparsification of a input P-bit A_0 in an 8-bit $\times$ 8-bit multiplier. Lines are COPY gates and circles are P-bits.

multiplier. In this case, we add nine auxiliary P-bits to reduce the number of connections to five, starting from the initial configuration with 25 connections.

- [1] S. Arora and B. Barak, *Computational Complexity: A Modern Approach* (Cambridge University Press, Cambridge, 2009).
- [2] M. A. Nielsen and I. L. Chuang, *Quantum Computation and Quantum Information: 10th Anniversary Edition* (Cambridge University Press, Cambridge, 2010).
- [3] P. W. Shor, Polynomial-time algorithms for prime factorization and discrete logarithms on a quantum computer, *SIAM J. Comput.* **26**, 1484 (1997).
- [4] L. K. Grover, in *Proceedings of the Twenty-Eighth Annual ACM Symposium on Theory of Computing*, STOC '96 (Association for Computing Machinery, New York, NY, USA, 1996), p. 212.
- [5] H. Aghaee Rad, T. Ainsworth, R. N. Alexander, B. Altieri, M. F. Askarani, R. Baby, L. Banchi, B. Q. Baragiola, J. E. Bourassa, R. S. Chadwick *et al.*, Scaling and networking a modular photonic quantum computer, *Nature* **638**, 912 (2025).
- [6] Google Quantum AI and Collaborators, Quantum error correction below the surface code threshold, *Nature* **638**, 920 (2025).
- [7] D. Gao, D. Fan, C. Zha, J. Bei, G. Cai, J. Cai, S. Cao, F. Chen, J. Chen, K. Chen *et al.*, Establishing a new benchmark in quantum computational advantage with 105-qubit Zuchongzhi 3.0 processor, *Phys. Rev. Lett.* **134**, 090601 (2025).
- [8] S. Kirkpatrick, C. D. Gelatt, and M. P. Vecchi, Optimization by simulated annealing, *Science* **220**, 671 (1983).
- [9] D. H. Ackley, G. E. Hinton, and T. J. Sejnowski, A learning algorithm for Boltzmann machines, *Cogn. Sci.* **9**, 147 (1985).
- [10] Y. Fu and P. W. Anderson, Application of statistical mechanics to NP-complete problems in combinatorial optimisation, *J. Phys. A: Math. Gen.* **19**, 1605 (1986).
- [11] N. Mohseni, P. L. McMahon, and T. Byrnes, Ising machines as hardware solvers of combinatorial optimization problems, *Nat. Rev. Phys.* **4**, 363 (2022).

- [12] D. J. Earl and M. W. Deem, Parallel tempering: Theory, applications, and new perspectives, *Phys. Chem. Chem. Phys.* **7**, 3910 (2005).
- [13] H. Goto, K. Tatsumura, and A. R. Dixon, Combinatorial optimization by simulating adiabatic bifurcations in nonlinear Hamiltonian systems, *Sci. Adv.* **5**, eaav2372 (2019).
- [14] H. Goto, K. Endo, M. Suzuki, Y. Sakai, T. Kanao, Y. Hamakawa, R. Hidaka, M. Yamasaki, and K. Tatsumura, High-performance combinatorial optimization based on classical mechanics, *Sci. Adv.* **7**, eabe7953 (2021).
- [15] Y. Yamamoto, K. Aihara, T. Leleu, K.-i. Kawarabayashi, S. Kako, M. Fejer, K. Inoue, and H. Takesue, Coherent Ising machines—Optical neural networks operating at the quantum limit, *Npj Quantum Inf.* **3**, 49 (2017).
- [16] B. Sutton, R. Faria, L. A. Ghantasala, R. Jaiswal, K. Y. Camsari, and S. Datta, Autonomous probabilistic coprocessing with petaflips per second, *IEEE Access* **8**, 157238 (2020).
- [17] W. A. Borders, A. Z. Pervaiz, S. Fukami, K. Y. Camsari, H. Ohno, and S. Datta, Integer factorization using stochastic magnetic tunnel junctions, *Nature* **573**, 390 (2019).
- [18] J. Si, S. Yang, Y. Cen, J. Chen, Y. Huang, Z. Yao, D.-J. Kim, K. Cai, J. Yoo, X. Fong, and H. Yang, Energy-efficient superparamagnetic Ising machine and its application to traveling salesman problems, *Nat. Commun.* **15**, 3457 (2024).
- [19] K. S. Woo, J. Kim, J. Han, W. Kim, Y. H. Jang, and C. S. Hwang, Probabilistic computing using $\text{Cu}_{0.1}\text{Te}_{0.9}/\text{HfO}_2/\text{Pt}$ diffusive memristors, *Nat. Commun.* **13**, 5762 (2022).
- [20] N. A. Aadit, A. Grimaldi, M. Carpentieri, L. Theogarajan, J. M. Martinis, G. Finocchio, and K. Y. Camsari, Massively parallel probabilistic computing with sparse Ising machines, *Nat. Electron.* **5**, 460 (2022).
- [21] N. Onizawa, K. Nishino, S. C. Smithson, B. H. Meyer, W. J. Gross, H. Yamagata, H. Fujita, and T. Hanyu, A design framework for invertible logic, *IEEE Trans. Comput.-Aided Des. Integr. Circuits Syst.* **40**, 655 (2021).
- [22] M. M. H. Sajeeb, N. A. Aadit, S. Chowdhury, T. Wu, C. Smith, D. Chinmay, A. Raut, K. Y. Camsari, C. Delacour, and T. Srimani, Scalable connectivity for Ising machines: Dense to sparse, *Phys. Rev. Appl.* **24**, 014005 (2025).
- [23] S. Nikhar, S. Kannan, N. A. Aadit, S. Chowdhury, and K. Y. Camsari, All-to-all reconfigurability with sparse and higher-order Ising machines, *Nat. Commun.* **15**, 8977 (2024).
- [24] S. Ruder, An overview of gradient descent optimization algorithms, *ArXiv:1609.04747*.
- [25] K. Y. Camsari, R. Faria, B. M. Sutton, and S. Datta, Stochastic p-bits for invertible logic, *Phys. Rev. X* **7**, 031014 (2017).
- [26] A. Z. Pervaiz, B. M. Sutton, L. A. Ghantasala, and K. Y. Camsari, Weighted p-bits for FPGA implementation of probabilistic circuits, *IEEE Trans. Neural Netw. Learn. Syst.* **30**, 1920 (2019).
- [27] D. J. Wales and P. K. Doye, Global optimization by basin-hopping and the lowest energy structures of Lennard-Jones clusters containing up to 110 atoms, *J. Phys. Chem. A* **101**, 5111 (1997).
- [28] M. K. Bashar and N. Shukla, Designing Ising machines with higher order spin interactions and their application in solving combinatorial optimization, *Sci. Rep.* **13**, 9558 (2023).
- [29] C. Bybee, D. Kleyko, D. E. Nikonov, A. Khosrowshahi, B. A. Olshausen, and F. T. Sommer, Efficient optimization with higher-order Ising machines, *Nat. Commun.* **14**, 6033 (2023).
- [30] Y. He, S. Luo, C. Fang, and G. Liang, Direct design of ground-state probabilistic logic using many-body interactions for probabilistic computing, *Sci. Rep.* **14**, 15076 (2024).
- [31] J. Kaiser, R. Faria, K. Camsari, and S. Datta, Probabilistic circuits for autonomous learning: A simulation study, *Front. Comput. Neurosci.* **14** (2020).
- [32] D. P. Rosin, D. Rontani, and D. J. Gauthier, Ultrafast physical generation of random numbers using hybrid Boolean networks, *Phys. Rev. E* **87**, 040902(R) (2013).
- [33] A. Rockovich, G. Lafyatis, and D. Gauthier, Simple-sparse-Ising-machine, 2025, <https://github.com/quantinfo/Simple-sparse-Ising-machine>
- [34] A. Rockovich, G. Lafyatis, and D. Gauthier, All project data, https://figshare.com/articles/dataset/All_project_data/29145572 (2025).